

◆ WEEK 2 — SESSION 4

COMPUTE LAYER (EC2 + INSTANCE TYPES + AWS LAMBDA)

◆ GITHUB LAB PACK — STUDENT PRACTICALS

◆ GITHUB REPOSITORY STRUCTURE

```
◆ compute-layer-lab/
├── ◆ README.md
├── ◆ labs/
│   ├── ◆ lab-01-launch-ec2-instance.md
│   ├── ◆ lab-02-instance-types-analysis.md
│   ├── ◆ lab-03-security-group-control.md
│   ├── ◆ lab-04-introduction-to-lambda.md
│   └── ◆ lab-05-ec2-vs-lambda-comparison.md
├── ◆ lambda-code/
│   ├── ◆ hello-world-lambda.py
│   └── ◆ event-response-example.py
├── ◆ screenshots/
│   └── ◆ upload-evidence-here/
├── ◆ architecture-notes/
│   ├── ◆ compute-vs-serverless.md
│   └── ◆ scaling-concepts.md
└── ◆ validation-checklists/
    ├── ◆ ec2-launch-checklist.md
    ├── ◆ security-group-checklist.md
    ├── ◆ lambda-checklist.md
    └── ◆ governance-thinking-checklist.md
```

◆ README.md — STUDENT STARTING GUIDE

◆ PURPOSE

This lab introduces students to AWS compute services and infrastructure thinking.

You will learn how AWS runs:

- ◆ applications
- ◆ servers
- ◆ workloads
- ◆ automation systems

This is your first step into real compute engineering.

◆ LEARNING PHILOSOPHY

This lab is NOT about:

- ✗ memorizing buttons
- ✗ copying tutorials
- ✗ launching random servers

This lab IS about:

- ◆ infrastructure awareness
- ◆ cloud engineering thinking
- ◆ security-conscious deployment
- ◆ understanding compute decisions

◆ REQUIRED AWS SERVICES

Students will work with:

- ◆ Amazon EC2
- ◆ Security Groups
- ◆ Instance Types
- ◆ AWS Lambda

◆ IMPORTANT RULES

- ◆ Never leave unnecessary ports open
- ◆ Never expose infrastructure carelessly
- ◆ Always validate access controls
- ◆ Always stop unused resources after labs
- ◆ Think about cost and security together

◆ LAB 01 — LAUNCH YOUR FIRST EC2 INSTANCE

■ `labs/lab-01-launch-ec2-instance.md`

◆ OBJECTIVE

Launch and access your first AWS virtual server.

◆ TASKS

- ◆ Open AWS EC2 Console
- ◆ Launch Linux EC2 instance
- ◆ Choose Amazon Linux AMI
- ◆ Select free-tier eligible instance
- ◆ Create key pair
- ◆ Configure Security Group
- ◆ Launch instance

◆ CONNECT TO INSTANCE

Students must:

- ◆ Copy public IP
- ◆ Connect using EC2 Instance Connect OR SSH
- ◆ Verify successful server access

◆ VALIDATION CHECK

- ✓ EC2 instance running
- ✓ Public IP assigned
- ✓ SSH connection successful
- ✓ Security Group configured properly

◆ THINKING MODEL

- ◆ Why does the server need a public IP?
- ◆ What risks exist if SSH is open to everyone?
- ◆ What happens if key pairs are lost?
- ◆ Why is compute considered infrastructure?

◆ LAB 02 — INSTANCE TYPES ANALYSIS

■ `labs/lab-02-instance-types-analysis.md`

◆ OBJECTIVE

Understand how compute resources affect workload performance and cost.

◆ TASKS

Research and compare:

Instance Family	Primary Use
General Purpose	Balanced workloads
Compute Optimized	High CPU processing
Memory Optimized	Large RAM workloads
Storage Optimized	High-speed storage
GPU Instances	AI / ML workloads

◆ STUDENT ACTIVITY

Match workloads to instance types:

- ◆ Banking application
 - ◆ AI training system
 - ◆ Small business website
 - ◆ Real-time analytics platform
 - ◆ Large storage logging system
-

◆ VALIDATION CHECK

- ✓ Student identifies correct instance categories
 - ✓ Student explains workload reasoning
 - ✓ Student understands cost vs performance tradeoff
-

◆ THINKING MODEL

- ◆ Why is overprovisioning dangerous?
- ◆ Why is underpowered compute risky?
- ◆ How do businesses optimize infrastructure costs?

◆ LAB 03 — SECURITY GROUP CONTROL

■ labs/lab-03-security-group-control.md

◆ OBJECTIVE

Understand network-level access control for compute infrastructure.

◆ TASKS

Students must:

- ◆ Review inbound rules
 - ◆ Identify open ports
 - ◆ Restrict unnecessary exposure
 - ◆ Allow SSH access securely
 - ◆ Test internet accessibility
-

◆ REQUIRED TESTS

Try:

- ◆ SSH access ✓
 - ◆ Browser access ✓ / ✗ depending on configuration
 - ◆ Unauthorized port access ✗
-

◆ VALIDATION CHECK

- ✓ Only required ports open
 - ✓ Security Groups functioning correctly
 - ✓ Exposure minimized
-

◆ THINKING MODEL

- ◆ What services are exposed publicly?
 - ◆ How do attackers scan infrastructure?
 - ◆ What happens if all ports are open?
-

◆ LAB 04 — INTRODUCTION TO AWS LAMBDA

📄 labs/lab-04-introduction-to-lambda.md

◆ OBJECTIVE

Deploy and execute your first serverless function.

◆ TASKS

Students must:

- ◆ Open AWS Lambda Console
 - ◆ Create Lambda Function
 - ◆ Select Python runtime
 - ◆ Paste sample code
 - ◆ Deploy function
 - ◆ Execute test event
-

◆ SAMPLE LAMBDA CODE

📄 lambda-code/hello-world-lambda.py

```
def lambda_handler(event, context):  
    return {  
        'statusCode': 200,  
        'body': 'Hello from AWS Lambda'  
    }
```

◆ VALIDATION CHECK

- ✓ Lambda function created
- ✓ Function deployed successfully

- ✓ Test event executed
- ✓ Successful output returned

◆ THINKING MODEL

- ◆ Why is Lambda called “serverless”?
- ◆ What infrastructure does AWS manage for you?
- ◆ When is Lambda better than EC2?

◆ LAB 05 — EC2 VS LAMBDA COMPARISON

📄 labs/lab-05-ec2-vs-lambda-comparison.md

◆ OBJECTIVE

Develop architectural thinking between traditional compute and serverless compute.

◆ STUDENT TASK

Complete comparison table:

EC2	Lambda
Full server management	Serverless
Long-running workloads	Event-driven execution
Manual scaling	Automatic scaling
OS-level control	Managed infrastructure

◆ REQUIRED ANALYSIS

Students must explain:

- ◆ When EC2 is preferred
- ◆ When Lambda is preferred
- ◆ Cost differences

- ◆ Operational differences
 - ◆ Security implications
-

◆ VALIDATION CHECK

- ✓ Student understands compute models
 - ✓ Student understands workload alignment
 - ✓ Student understands operational tradeoffs
-

◆ SCREENSHOT REQUIREMENTS

Students must upload evidence to:

📁 `screenshots/upload-evidence-here/`

Required screenshots:

- ◆ Running EC2 instance
 - ◆ Security Group rules
 - ◆ Successful SSH connection
 - ◆ Lambda function deployed
 - ◆ Lambda test success output
-

◆ FINAL GOVERNANCE THINKING

Modern cloud engineering is not about launching servers.

It is about:

- ◆ choosing correct infrastructure
 - ◆ minimizing risk
 - ◆ controlling exposure
 - ◆ optimizing performance
 - ◆ managing cost intelligently
-

◆ FINAL PRINCIPLE

“The best cloud engineers do not deploy randomly — they design intentionally.”